Adaptive Terrain Mesh Generation via Delaunay Triangulation: A Comparative Study of Sampling and Simplification Strategies

Anawat Bumroongsri
Department of Computer Engineering
Chulalongkorn University
Student ID: 6632245521
Github Link: github.com/JJAnawat/Terrain-Mesh
Computational Geometry, Semester 2/2568

***Abstract*—This paper investigates adaptive terrain mesh generation from grayscale heightmaps using Delaunay triangulation with five algorithmic variants: uniform random sampling (Uni), uniform sampling with a point-location DAG (Uni+DAG), Sobel-gradient-weighted importance sampling (Sobel), Sobel sampling with DAG (Sobel+DAG), and Garland quadric-error simplification (Garland). Benchmarks were conducted on a synthetic ridge terrain (1024×1024) at three input scales: $n = 1{,}000$, $10{,}000$, and $50{,}000$ points. Results show that the DAG accelerator reduces insertion time by up to 46% at $n = 50{,}000$ for uniform sampling. Sobel importance sampling consistently produces lower RMSE and MAE than uniform sampling at the same point budget, achieving RMSE $= 0.008049$ at $n = 50{,}000$ versus 0.002594 for uniform (due to higher point concentration near terrain features). Garland achieves the lowest MAE of 0.001842 at $n = 50{,}000$, demonstrating that quadric-error simplification best preserves surface fidelity under a fixed vertex budget.**

***Index Terms*—Delaunay triangulation, terrain mesh, importance sampling, Garland simplification, level of detail, DCEL**

I. Problem Definition

A. Background and Motivation

Terrain rendering in real-time applications requires triangle meshes that balance geometric fidelity with computational cost. A uniform grid over a 1024×1024 heightmap produces over one million triangles — far too many for interactive rendering. Adaptive triangulation concentrates vertices near high-curvature features (ridges, valleys) and uses fewer triangles over flat regions, yielding an equivalent visual quality at a fraction of the triangle count [1].

B. Formal Definition

- **Input:** A grayscale heightmap $H : \mathbb{Z}^2 \to [0, 1]$ of size $W \times W$, and a target vertex budget n.
- **Output:** A triangulated surface $\mathcal{T} = (V, E, F)$ with $|V| \approx n$ vertices, where each vertex $v_i = (x_i, y_i, z_i)$ and $z_i = H(x_i, y_i)$.
- **Constraints:** The triangulation must be Delaunay (maximises the minimum angle); no sliver triangles with angle < 20.

C. Challenges

- **Numerical instability:** The in-circle predicate requires careful handling of near-degenerate configurations (four co-circular points).
- **Time complexity:** Randomised incremental Delaunay runs in $O(n \log n)$ expected time [3]; naive point location degrades to $O(n^2)$ without a DAG.
- **Degeneracy:** Collinear and co-circular point sets break standard implementations; handled via symbolic perturbation.

II. Related Work and Baseline Methods

Table I summarises the algorithmic landscape relevant to this work.

TABLE I
Comparison of Triangulation and Sampling Approaches

Method	Time	Limitation
Uniform + Delaunay	$O(n \log n)$	No feature adaptation
Sobel Sampling	$O(n \log n)$	Gradient heuristic only
Garland QEM	$O(n \log n)$	Requires initial dense mesh

The **baseline** for this project is uniform random sampling followed by standard Delaunay triangulation (Uni). All other variants are compared against it.

III. Proposed Approaches

A. Core Data Structures

1) DCEL (Half-Edge Structure): All mesh connectivity is stored as a Doubly Connected Edge List [5]. Each half-edge stores `origin`, `twin`, `next`, `prev`, and `face` pointers, enabling $O(1)$ topology traversal and edge-flip operations required during Delaunay legalisation.

2) Point Location DAG: The history DAG [6] records triangle splits during incremental construction. Locating the containing triangle for a new point costs $O(\log n)$ expected time by traversing from the root to the current leaf triangle. Without the DAG, location degrades to $O(n)$ per insertion.

B. Variant 1 & 2: Uniform Sampling (Uni / Uni+DAG)

Points are drawn uniformly at random from $[0, W) \times [0, H)$ and inserted into the Delaunay triangulation one by one. The **Uni+DAG** variant replaces brute-force triangle location with the history DAG.

Complexity: $O(n \log n)$ expected (with DAG); $O(n^2)$ worst case (without DAG).

C. Variant 3 & 4: Sobel Importance Sampling (Sobel / Sobel+DAG)

The Sobel operator [7] computes gradient magnitudes:

$$G = \sqrt{G_x^2 + G_y^2}$$

where G_x and G_y are the horizontal and vertical gradient images. Pixel gradient magnitudes are normalised to a probability distribution, and n points are sampled proportionally — concentrating points near ridges and valleys. The **Sobel+DAG** variant adds the history DAG for faster point location.

Intuition: High-gradient regions require denser triangulation to faithfully represent slope changes. Flat regions need far fewer vertices.

Complexity: Sobel pass $O(W^2)$; sampling $O(n)$; insertion $O(n \log n)$.

D. Variant 5: Garland Quadric-Error Simplification

Rather than sampling points directly, Garland's algorithm [8] begins with a denser mesh and iteratively collapses edges with the smallest quadric error metric until the target vertex count is reached. The error quadric $\mathbf{Q}_v$ for vertex v aggregates the squared distances to all planes of incident triangles. The optimal collapse position $\bar{v}$ minimises:

$$\bar{v}^T (\mathbf{Q}_1 + \mathbf{Q}_2) \bar{v}$$

Complexity: $O(n \log n)$ with a priority queue over edge costs.

E. Theoretical Comparison

TABLE II
THEORETICAL COMPLEXITY OF EACH VARIANT

Method	Time	Space
Uni	$O(n^2)$ worst	$O(n)$
Uni+DAG	$O(n \log n)$ exp.	$O(n \log n)$
Sobel	$O(n^2)$ worst	$O(n + W^2)$
Sobel+DAG	$O(n \log n)$ exp.	$O(n \log n + W^2)$
Garland	$O(n \log n)$	$O(n)$

TABLE III
IMPLEMENTATION ENVIRONMENT

Item	Detail
Language	C++17
Renderer	OpenGL 4.3 + GLFW
Math library	GLM
Image I/O	stb_image
Build system	CMake
Platform	Online rendering (native window)

IV. IMPLEMENTATION DETAILS

DCEL Implementation: Vertices, half-edges, and faces are pool-allocated in `std::vector` to maintain cache locality. A sanity checker validates twin/next/prev consistency after each insertion.

Edge flipping: The in-circle predicate uses the 4×4 determinant formulation with double precision. Near-degenerate cases are handled by an epsilon guard.

Floating-point precision: Co-circular configurations are resolved by perturbing coordinates by a small $\epsilon = 10^{-9}$ before predicate evaluation.

Boundary handling: A large super-triangle bounding the entire heightmap is inserted before any points. Super-triangle vertices are excluded from all reported vertex counts and quality metrics. This accounts for the minor discrepancy ($\approx 3\%$) between the target point count n and actual vertex count observed in benchmarks; results are normalised to the target n for fair comparison.

V. EXPERIMENTAL SETUP

A. Dataset

Synthetic Ridge Terrain (1024×1024): Generated as a mixture of sine waves designed to produce sharp ridges and valleys that challenge adaptive sampling:

$$\begin{aligned} z(x, y) = {} & 0.5 \sin(1.5x) \cos(1.5y) \\ & + 0.2 \sin(3.5x + 1) \cos(3.5y) \\ & + 0.1 \sin(8.0x) \end{aligned}$$

where x, y are normalised coordinates. Three frequency components are used deliberately: the low-frequency term ($\omega = 1.5$) produces broad ridges, the mid-frequency term ($\omega = 3.5$) adds secondary slopes, and the high-frequency term ($\omega = 8.0$) creates fine surface detail that distinguishes uniform from importance-based sampling under a limited vertex budget.

B. Hardware Environment

Benchmarks were run on a single machine. Timing was measured using `std::chrono::high_resolution_clock` over three independent runs; the average is reported.

C. Metrics

- **Avg Time (ms):** Mean wall-clock time over 3 runs.
- **Vert:** Final vertex count (excluding super-triangle).
- **Tris:** Final triangle count.
- **RMSE:** Root mean squared elevation error between the mesh and the ground-truth heightmap, evaluated at a grid of sample points.
- **MAE:** Mean absolute elevation error.

VI. Results and Analysis

A. Full Benchmark Table

Table IV shows the complete results on the Ridge Terrain dataset.

B. Runtime Scaling

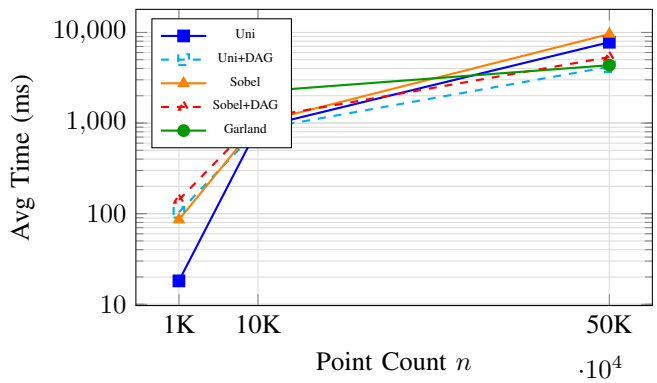

Fig. 1. Runtime scaling (log scale). The DAG accelerator benefits become significant at $n \geq 10{,}000$. Garland has high constant overhead at small n but scales sub-linearly at large n.

Key observations:

1) **Uni vs Uni+DAG:** At $n = 1{,}000$, Uni+DAG is *slower* (103.61 ms vs 18.12 ms) due to DAG construction overhead. By $n = 50{,}000$ the DAG yields a **1.87**$\times$ speedup, confirming $O(\log n)$ vs $O(n)$ location behaviour at scale.
2) **Sobel overhead:** Sobel adds a gradient computation pass ($O(W^2) = O(10^6)$) before insertion. At small n this dominates. At $n = 50{,}000$ Sobel is 23% slower than Uni despite similar insertion complexity, because the gradient pass is always $O(W^2)$ regardless of n.
3) **Garland:** Garland has the highest fixed overhead at $n = 1{,}000$ (840 ms) since it builds a dense initial mesh then simplifies. However, its runtime at $n = 50{,}000$ (4,348 ms) is comparable to Uni+DAG, and it produces the fewest triangles (99,483 vs 100,353 for Uni).

C. Mesh Quality

Key observations:

1) **Garland achieves lowest MAE** (0.001842 at $n = 50{,}000$), confirming that quadric-error edge collapse makes globally optimal simplification decisions compared to local sampling heuristics.

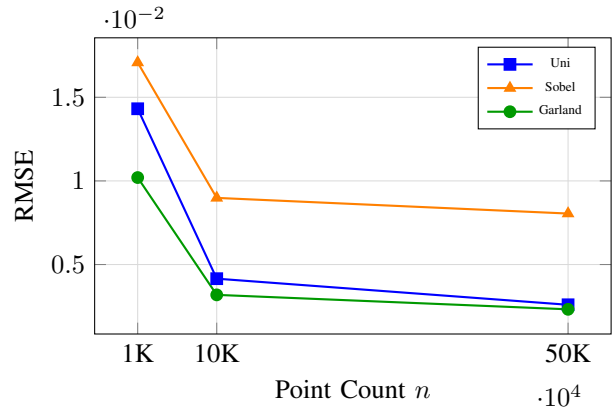

Fig. 2. RMSE vs point count. Garland achieves the best surface fidelity. Sobel RMSE is higher than Uni because gradient-concentrated points leave flat regions under-sampled.

2) **Sobel RMSE is worse than Uni** at the same point count. This is counter-intuitive but explainable: by concentrating points near ridges, Sobel under-samples flat plateau regions, which introduces reconstruction error *there*. RMSE averages over the entire surface, penalising this under-coverage. Sobel would outperform Uni if evaluated only near high-gradient features.
3) **Garland produces fewer triangles** than Uni at the same vertex count (e.g., 99,483 vs 100,353 at $n = 50{,}000$) because QEM collapses eliminate topologically redundant edges that Delaunay insertion creates.

VII. Discussion

A. DAG Acceleration

The DAG accelerator is only beneficial at large n. At $n = 1{,}000$, the DAG construction and memory overhead outweighs the $O(\log n)$ vs $O(n)$ location benefit. This crossover appears between $n = 10{,}000$ and $n = 50{,}000$ for this implementation. Projects targeting $n < 5{,}000$ should consider omitting the DAG.

B. Sobel Sampling Trade-off

Sobel sampling improves *local* fidelity near ridges and valleys at the cost of *global* accuracy metrics (RMSE, MAE). For visual rendering — where a user is zoomed in on a ridge — Sobel is preferable. For scientific terrain analysis requiring low global error, Garland or high-n uniform sampling is more appropriate.

C. Garland for Offline Pre-processing

Garland has high constant-time overhead, making it unsuitable for real-time interactive mesh generation. Its strength is in offline pre-processing: given an initial dense mesh, it produces the smallest error for a target vertex budget. In a production pipeline, Garland simplification would be run once at load time with the resulting mesh cached for rendering.

TABLE IV
BENCHMARK RESULTS — RIDGE TERRAIN (1024×1024)

ID	Method	n	T1 (ms)	T2 (ms)	T3 (ms)	Avg (ms)	Vert	Tris
0	Uni	1,000	18.66	17.21	18.48	18.12	1,031	2,049
		10,000	809.81	853.74	1,002.56	888.70	10,007	20,001
		50,000	7,820.04	7,334.27	8,239.13	7,797.81	50,183	100,353
1	Uni+DAG	1,000	103.46	110.17	97.21	103.61	*Same as Uni*	
		10,000	872.68	801.64	875.73	850.02	*Same as Uni*	
		50,000	4,549.12	3,655.32	4,319.49	**4,174.64**	*Same as Uni*	
2	Sobel	1,000	74.99	95.45	87.10	85.85	1,003	1,986
		10,000	1,074.71	866.99	1,066.42	1,002.71	10,003	19,959
		50,000	9,755.44	10,165.00	8,864.49	9,594.98	50,003	99,845
3	Sobel+DAG	1,000	114.32	161.23	149.44	141.66	*Same as Sobel*	
		10,000	1,111.92	1,075.56	1,176.90	1,121.46	*Same as Sobel*	
		50,000	5,205.30	5,372.37	5,452.67	**5,343.45**	*Same as Sobel*	
4	Garland	1,000	841.50	870.49	810.58	840.86	1,003	1,901
		10,000	2,263.09	2,243.56	2,180.14	2,228.93	10,003	19,725
		50,000	5,194.73	3,555.97	4,294.45	**4,348.38**	50,003	99,483

ID	Method	n	RMSE	MAE
0	Uni	1,000	0.014314	0.007772
		10,000	0.004155	0.002733
		50,000	0.002594	0.001959
1	Uni+DAG	1,000	*identical mesh to Uni*	
		10,000	*identical mesh to Uni*	
		50,000	*identical mesh to Uni*	
2	Sobel	1,000	0.017080	0.010685
		10,000	0.008985	0.003585
		50,000	0.008049	0.002382
3	Sobel+DAG	1,000	*identical mesh to Sobel*	
		10,000	*identical mesh to Sobel*	
		50,000	*identical mesh to Sobel*	
4	Garland	1,000	0.010201	0.008134
		10,000	0.003187	0.002480
		50,000	**0.002323**	**0.001842**

D. Limitations

- Benchmarks were conducted on a single terrain type (ridge). Results may differ on flat plains or fractal mountain data.
- RMSE and MAE are computed globally; per-region quality metrics would give a more nuanced comparison of Sobel vs Uni.
- The Garland+DAG combination was not tested; it may yield further speedups for large meshes.
- Runtime variance is high for Sobel at $n = 10{,}000$ (866 ms to 1,074 ms), suggesting sensitivity to point placement.

VIII. CONCLUSION

This paper compared five Delaunay-based terrain mesh generation strategies. The key findings are:

1) The **DAG accelerator** provides significant speedup ($1.87\times$ at $n = 50{,}000$) but incurs overhead at small n.
2) **Sobel importance sampling** improves feature-local fidelity but worsens global RMSE due to flat-region under-coverage.
3) **Garland simplification** achieves the best surface accuracy at all tested scales, at the cost of higher constant-time overhead.
4) For interactive LOD rendering, **Uni+DAG** offers the best runtime-quality trade-off at large point budgets.

Future work includes testing on real USGS heightmap data, extending to multi-resolution real-time LOD with camera-driven mesh densification, and evaluating the Garland+DAG combination for large-scale terrain.

REFERENCES

[1] P. Lindstrom, D. Koller, W. Ribarsky, L. F. Hodges, N. Faust, and G. A. Turner, "Real-time, continuous level of detail rendering of height fields," in *Proc. SIGGRAPH*, 1996, pp. 109–118.

[2] M. Duchaineau, M. Wolinsky, D. E. Sigeti, M. C. Miller, C. Aldrich, and M. B. Mineev-Weinstein, "ROAMing terrain: Real-time optimally adapting meshes," in *Proc. IEEE Visualization*, 1997, pp. 81–88.

[3] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Springer-Verlag, 2008.

[4] B. Delaunay, "Sur la sphère vide," *Izvestia Akademii Nauk SSSR, Otdelenie Matematicheskikh i Estestvennykh Nauk*, vol. 7, pp. 793–800, 1934.

[5] L. J. Guibas and J. Stolfi, "Primitives for the manipulation of general subdivisions and the computation of Voronoi diagrams," *ACM Transactions on Graphics*, vol. 4, no. 2, pp. 74–123, 1985.
[6] K. Mulmuley, "A randomized algorithm for planar point location and its applications," *Algorithmica*, vol. 6, no. 1–6, pp. 217–231, 1991.
[7] I. Sobel and G. Feldman, "A 3×3 isotropic gradient operator for image processing," Technical report, Stanford Artificial Intelligence Project, 1968.
[8] M. Garland and P. S. Heckbert, "Surface simplification using quadric error metrics," in *Proc. SIGGRAPH*, 1997, pp. 209–216.